

# Problem Statement: Product Verification System

## 1. Objective

To design and build a robust system that allows for the bulk import of product data, facilitates on-the-spot verification of individual products in a warehouse, and generates comprehensive reports on all verification activities.

## 2. Background

In our logistics and warehouse operations, maintaining accurate product information is crucial for efficient inventory management, quality control, and ensuring compliance with expiry regulations. We handle a vast and diverse range of products, and we require a streamlined, reliable system to manage and verify their lifecycle data, specifically manufacturing and expiry dates.

## 3. Core Features & User Stories

The system should be built around the following core functionalities:

### 3.1. Bulk Product Data Ingestion

- **User:** Warehouse Manager
- **Story:** As a warehouse manager, I need to upload a CSV file containing product data to quickly and efficiently populate the system with new or incoming inventory information.
- **Details:**
  - The CSV file will contain the following columns:
    - **WID:** A unique Warehouse ID for each individual physical item. This is the primary identifier.
    - **EAN:** The European Article Number, which is the product's barcode ID.
    - **Manufacturing\_Date:** The date the product was manufactured.
    - **Expiry\_Date:** The date the product expires.
  - A single product type (identified by its **EAN**) can have multiple entries in the system, each with a unique **WID** and potentially different manufacturing and expiry dates.
  - All data from the uploaded file must be stored in a persistent database.

### 3.2. On-the-Floor Product Validation

- **User:** Warehouse Operator
- **Story:** As a warehouse operator, I need a simple, fast interface to verify a product's details while I am physically handling it on the warehouse floor.
- **Details:**
  - The interface should allow me to input a product's unique **WID** ( through barcode ).
  - The system must also provide a way to capture a picture of the physical product being inspected.
  - Upon submission, the system should instantly display the **EAN**, **Manufacturing\_Date**, and **Expiry\_Date** associated with the entered **WID**.
  - This allows me to visually compare the data in the system against the physical product's label to confirm accuracy.
  - The system should log every validation event (e.g., which user checked which WID and when).

### 3.3. Verification Reporting

- **User:** Quality Assurance Manager
- **Story:** As a quality assurance manager, I need to be able to generate reports on all verification activities to track compliance and identify potential issues.
- **Details:**
  - The reporting interface should allow me to select a start date and an end date.
  - The system must generate and display a report of all verification activities that occurred within that specified date range.

## 4. Key Considerations & Constraints

- **Scalability & Performance:** The system must be designed to handle very large CSV files (potentially containing millions of records) without failure. The validation and reporting features must remain fast and responsive, even as the database grows and the report's date range expands.
- **Usability:** The validation page for the warehouse operator should be straightforward and optimized for quick, repetitive use, potentially on a mobile or handheld device.
- **Data Integrity:** The uniqueness of the **WID** is critical and must be strictly enforced throughout the system.

## 5. Your Task

Your task is to build a functional, end-to-end application that meets all the requirements outlined in this document.

**Deliverables:**

1. **A Working Application:** An application that fully implements all core features described in section 3.
2. **Source Code:** Provide access to the complete, well-documented, and production-ready source code.
3. **Setup & Deployment Instructions:** Clear documentation on how to set up the development environment, build the application, and deploy it.
4. **Presentation & Code Walkthrough:** Be prepared to demonstrate your live application and walk us through your code. You should be ready to discuss your architectural decisions, technology stack choices, data schema, and how you addressed the key considerations (scalability, usability, data integrity).

## 6. Non - Mandatory Good to Haves (Optional)

The below should be tried only if the main task has been completed.

- A user login page and a user creation page for admins.
  - 2 roles can be assigned by the admin during user creation.
- A role based access control for certain pages
  - Only the admin should be able to upload the file and download reports.
  - The operator should only be able to verify.
- Fetch manufacturing and expiry from the image automatically during image capture and validate against the data.

**Sample Upload CSV File:** [Link](#)

# Interview Challenge: Logistics Proof of Delivery (POD) App

## 1. Objective

Prototype a simple web/mobile application for delivery drivers. The app's goal is to replace manual, paper-based delivery confirmations.

## 2. Core Tasks

The application must accomplish the following:

1. **Scan AWB:** Use the device's camera to scan a package's Air Waybill (AWB) number (e.g., from a barcode or QR code).
2. **Capture Media:** Allow the driver to use the camera to take a photo or record a short video as proof of delivery.
3. **Cloud Storage:** The captured media (photo/video) must be associated with its AWB number and uploaded to a cloud storage solution.

## 3. Technology & Deliverable

- **Technology Freedom:** You are free to use any technology stack, libraries, or frameworks you are comfortable with (e.g., React, Angular, Vue, plain HTML/JS, or a mobile-native stack).
- **Deliverable:** Please provide a simple working prototype that demonstrates this flow **or** a high-level system design document that explains your proposed architecture and data model.

